Solution for Hlohovec Space Program

Usually, optimization problems are computationally hard and you are supposed to find heuristic ways to find some good-but-not-optimal solution.

This turns out to be a different case: optimizing flight costs for our rocket is reasonably easy and we can actually construct optimal solutions — along with proofs of their optimality. Hence, your submissions were scored according to the actually optimal paths. This also meant that it was possible to get a full score for the problem (even with some small leeway for rounding errors / lack of numerical precision in your calculations).

That being said, getting a full score from this problem is not the only possible goal. If we decide that our time is best spent on the rest of the problem set, it still doesn't make sense to abandon this problem completely. In order to get a non-zero partial score all we need to do is to print basically any sequence of $t-1$ points. And if we pick one that's not completely dumb — e.g., just print the $t-1$ points that divide the segment from start to finish into t equal parts — we should get about 25 points. Not bad for about five lines of code.

The rest of this text explains how to get the remaining 75. The outline of the solution:

1. Throw away the positions and describe the flight by its accelerations. The whole mission then collapses into **two vector equations**.
2. Observe that the cost function is **convex**, so there are no local minima to get stuck in.
3. Use Lagrange multipliers to turn the problem into an **unconstrained maximization in four real variables** — regardless of whether the mission takes 3 steps or 500.
4. Solve those four variables numerically, read the flight back out, and get a **proof of optimality** for free along the way.

Step 1: forget the positions, look at the accelerations

The output format asks for positions, but positions are a redundant description of the flight: consecutive positions give us velocities and consecutive velocities give us accelerations. It is much more convenient to treat the accelerations $a_0, a_1, \ldots, a_t$ as our unknowns and to compute the positions from them only at the very end.

From $v_{k+1} = v_k + a_k$ we immediately get

$$v_k = v_0 + \sum_{i<k} a_i,$$

and by summing those velocities ($p_k = p_0 + v_1 + \cdots + v_k$) we get

$$p_k = p_0 + k\,v_0 + \sum_{i=0}^{k-1} (k-i)\,a_i.$$

The second formula has a nice interpretation: an acceleration applied at step i keeps pushing us for all the remaining $k-i$ steps, so its effect on the position at time k is multiplied by $k-i$. Let's call the number

$$w_k := t - k$$

the **leverage** of step k: it says how much step k moves the final position p_t. Note that $w_t = 0$. This is because the very last acceleration happens at the exact moment we arrive at the target, so it cannot move us at all. Its only job is to fix the final velocity.

Now plug $k = t$ into the two formulas above. Everything the mission asks of us is precisely this:

$$\sum_{k=0}^{t} w_k\, a_k = c_1 := p_t - p_0 - t\, v_0, \qquad \sum_{k=0}^{t} a_k = c_2 := v_{t+1} - v_0.$$

That's it. **The entire mission is two linear vector equations in the accelerations.** The vectors c_1 and c_2 are computed directly from the input. In terms of ordinary real numbers we have $2(t+1)$ unknowns — up to 1002 of them — but only **four** scalar equations. The problem is massively underdetermined, and we get to pick the cheapest of all those solutions.

At this moment we'll make two additional observations:

- Any choice whatsoever of the "middle" accelerations $a_1, \ldots, a_{t-1}$ can be completed to a valid flight, because a_0 and a_t can then be computed from the two equations. (We'll use this in Step 5.)
- The mission can sometimes be flown completely for free. Such tests do appear in the test data and dealing with them will be the only potentially tricky part of the solution.

Step 2: the cost function is convex

We are minimizing $\sum_{k=0}^{t} h(a_k)$, where $h(a) = \max(\|a\| - a_{\text{free}},\, 0)^{1.5}$.

Recall that a function is *convex* if the line segment between any two points of its graph never dips below the graph. Convexity is the property that makes optimization problems tractable, because a convex function on a convex domain has **no local minima other than global ones** — so a local search can't get stuck in a bad valley.

We can easily show that our objective h and the constraints from Step 1 are all convex functions.

Step 3: Lagrange multipliers, or "four variables are enough"

Dualize the two vector constraints with multipliers $\lambda_1, \lambda_2 \in \mathbb{R}^2$:

$$L(a, \lambda) = \sum_{k=0}^{t} h(a_k) - \Big\langle \lambda_1, \sum_{k=0}^{t} w_k a_k - c_1 \Big\rangle - \Big\langle \lambda_2, \sum_{k=0}^{t} a_k - c_2 \Big\rangle.$$

Grouping by step with $g_k := w_k \lambda_1 + \lambda_2$ decouples the mission completely:

$$L(a, \lambda) = \langle \lambda_1, c_1 \rangle + \langle \lambda_2, c_2 \rangle + \sum_{k=0}^{t} \big(h(a_k) - \langle g_k, a_k \rangle \big).$$

Each step minimizes $h(a) - \langle g_k, a \rangle$ over $a \in \mathbb{R}^2$. Since $h(a)$ depends only on $\|a\|$, Cauchy–Schwarz implies a must point along g_k: $a = r\hat{g}_k$ where $r \geq 0$ and $\hat{g}_k = g_k / \|g_k\|$. With $G = \|g_k\|$, we minimize

$\varphi(r) = \max(r - a_{\text{free}}, 0)^{1.5} - rG$. For $r \geq a_{\text{free}}$, stationarity gives:

$$\varphi'(r) = 1.5\sqrt{r - a_{\text{free}}} - G = 0 \implies r = a_{\text{free}} + \tfrac{4}{9}G^2.$$

Substituting r back gives the minimum $-\left(a_{\text{free}}G + \tfrac{4}{27}G^3\right)$. This yields the dual function in closed form:

$$D(\lambda_1, \lambda_2) = \langle \lambda_1, c_1 \rangle + \langle \lambda_2, c_2 \rangle - \sum_{k=0}^{t} \left(a_{\text{free}} \|g_k\| + \tfrac{4}{27}\|g_k\|^3 \right), \qquad g_k = w_k\lambda_1 + \lambda_2.$$

By weak duality, $D(\lambda)$ is a lower bound on the optimal flight cost for any λ. As a pointwise infimum of affine functions, D is concave, smooth, and finite everywhere: maximizing it is an unconstrained 4-variable concave optimization problem.

To keep numerical variables well-conditioned (w_k reaches $t \leq 500$, which skews λ_1), reparametrize the affine line $g(w)$ by its endpoints $P := g(0) = \lambda_2$ and $Q := g(t) = t\lambda_1 + \lambda_2$, so $g_k = (1 - \frac{w_k}{t})P + \frac{w_k}{t}Q$.

Step 4: what does an optimal flight actually look like?

The marginal cost of burn $1.5\sqrt{\|a\| - a_{\text{free}}}$ increases with $\|a\|$. By strict convexity of the cost above a_{free}, spreading required momentum across many steps is always strictly cheaper than concentrating it into few burns. Hence every step burns; there is no sparsity or bang-bang control.

Even subproblem H1 ($a_{\text{free}} = 0$) has no closed form: the recovered acceleration $a_k = \frac{4}{9}\|g_k\|g_k$ remains nonlinear in g_k, requiring numerical dual optimization even without free allowance.

Step 5: reading a flight back out — and always getting a valid one

Given dual values, the recovered accelerations $a_k = (a_{\text{free}} + \frac{4}{9}\|g_k\|^2)\hat{g}_k$ only satisfy the mission equations at the exact dual optimum.

To guarantee exact feasibility, take the middle accelerations $a_1, \ldots, a_{t-1}$ from the dual formula and solve for a_0 and a_t using the mission equations. Since $w_0 = t$ is the unique step with leverage t and $w_t = 0$ is the unique step with leverage 0, this is a triangular solve:

$$a_0 = \left(c_1 - \sum_{k=1}^{t-1} w_k a_k \right) \Big/ t, \qquad a_t = c_2 - a_0 - \sum_{k=1}^{t-1} a_k.$$

Every iterate is thus an exactly feasible flight, and dual errors degrade the objective smoothly. Evaluating this flight's cost against the dual lower bound $D(P, Q)$ provides an explicit optimality certificate $\text{cost}(a) - D(P, Q) \geq 0$.

Step 6: solving the dual

When $\|g_k\| = 0$, the direction $\hat{g}_k$ is undefined. This degeneracy occurs whenever a mission can be flown completely for free (optimal $\lambda = 0$), or when the line $g(w)$ passes through the origin.

To remove degeneracy, add a quadratic regularizer $\frac{\delta}{2}\|a_k\|^2$ ($\delta > 0$) to the objective. For $G \leq \delta a_{\text{free}}$, the minimum sits below the kink at $r = G/\delta$ (so $a = g/\delta \to 0$ smoothly as $g \to 0$). For $G > \delta a_{\text{free}}$, stationarity

$1.5\sqrt{r - a_{\text{free}}} + \delta r - G = 0$ is a quadratic in $y = \sqrt{r - a_{\text{free}}}$:

$$\delta y^2 + 1.5y + (\delta a_{\text{free}} - G) = 0.$$

To avoid catastrophic cancellation when $\delta \to 0$, compute the positive root stably via the conjugate formula:

$$y = \frac{2(G - \delta a_{\text{free}})}{1.5 + \sqrt{1.5^2 - 4\delta(\delta a_{\text{free}} - G)}},$$

giving $r = a_{\text{free}} + y^2$.

Scale $c_1, c_2, a_{\text{free}}$ so accelerations are $O(1)$. Maximize D over the 4 variables using damped Newton (Levenberg–Marquardt, $O(t)$ per iteration) with continuation $\delta = 1 \to 10^{-12}$ warm-starting each stage. The certificate is evaluated using the unregularized dual.

Simpler options

- **Conic solver (SOCP):** The $3/2$ power is exactly second-order conic:

  $$e^{1.5} \le f \iff \exists\, z \ge 0 :\ z^2 \le e \ \text{ and } \ e^2 \le fz.$$

  Both are rotated second-order cones. Together with $\|a_k\| \le n_k$ and $e_k \ge n_k - a_{\text{free}}$, minimizing $\sum f_k$ is an exact second-order cone program, solved to $\approx 10^{-9}$ by solvers like Gurobi without piecewise-linear approximations. Endpoint pinning guarantees exact feasibility.
- **First-order methods:** Because the 1.5 exponent makes the cost C^1 smooth, generic structure-blind methods such as ADMM converge directly to the optimum.
- **Straight line:** Distributing displacement with constant velocity takes ~5 lines of code and scores ~25 points as a baseline.